Experiment 5

Aim: Write a programm to perform Left Factoring on a Grammar

Theory:

Left factoring is a grammar transformation used when two or more productions of the same non-terminal share a common prefix, such as A → $\alpha\beta_1$ | $\alpha\beta_2$ | ... , where α is the shared prefix. In this situation, a top-down parser looking at the input cannot decide which production to apply just by seeing the first few symbols, since more than one alternative starts the same way — this ambiguity in choice is what makes the grammar unsuitable for predictive (LL) parsing. Left factoring resolves this by factoring out the common prefix into a single production and deferring the choice to a new non-terminal: A → αA', A' → β_1 | β2 | ... , where A' is newly introduced. This delays the decision until enough of the input has been seen to distinguish between the alternatives, without changing the language the grammar generates. Along with removal of left recursion, left factoring is one of the standard grammar transformations applied before constructing a predictive parser in the syntax-analysis phase of a compiler.

Code:

```
#include <iostream>
#include <string>
#include <vector>
#include <map>
using namespace std;


vector<string> splitAlternatives(const string &rhs) {
   vector<string> parts;
   string current = "";
   for (char c : rhs) {
      if (c == '|') {
         parts.push_back(current);
         current = "";
      } else {
         current += c;
```

```
        }
    }
    parts.push_back(current);
    return parts;
}

string longestCommonPrefix(const vector<string> &group) {
    string prefix = group[0];
    for (size_t i = 1; i < group.size(); i++) {
        int j = 0;
        while (j < (int)prefix.length() && j < (int)group[i].length() && prefix[j] == group[i][j])
{
            j++;
        }
        prefix = prefix.substr(0, j);
    }
    return prefix;
}

void leftFactor(const string &production) {
    int arrowPos = production.find("->");
    string lhs = production.substr(0, arrowPos);
    string rhs = production.substr(arrowPos + 2);

    vector<string> alternatives = splitAlternatives(rhs);

    map<char, vector<string>> groups;
    vector<char> order;
```

```
for (const string &alt : alternatives) {
    char firstChar = alt.empty() ? '\0' : alt[0];
    if (groups.find(firstChar) == groups.end()) {
        order.push_back(firstChar);
    }
    groups[firstChar].push_back(alt);
}

vector<string> mainAlternatives;
vector<string> newProductions;
int counter = 1;

for (char c : order) {
    vector<string> &group = groups[c];
    if (group.size() == 1) {
        mainAlternatives.push_back(group[0]);
    } else {
        string prefix = longestCommonPrefix(group);
        string newNonTerminal = lhs + string(counter, '\'');
        counter++;
        mainAlternatives.push_back(prefix + newNonTerminal);

        string newProd = newNonTerminal + " -> ";
        for (size_t i = 0; i < group.size(); i++) {
            string suffix = group[i].substr(prefix.length());
            if (suffix.empty()) suffix = "e";
            newProd += suffix;
            if (i != group.size() - 1) newProd += " | ";
```

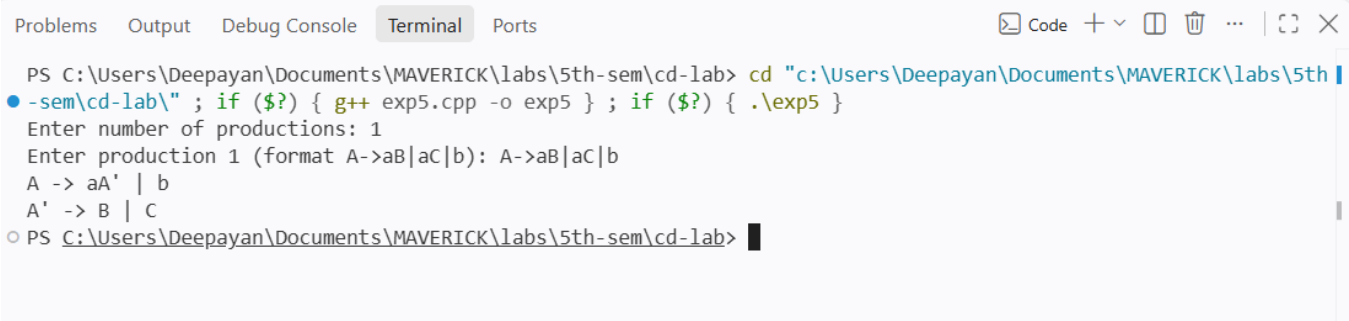
```
            }
            newProductions.push_back(newProd);
        }
    }

    cout << lhs << " -> ";
    for (size_t i = 0; i < mainAlternatives.size(); i++) {
        cout << mainAlternatives[i];
        if (i != mainAlternatives.size() - 1) cout << " | ";
    }
    cout << "\n";

    for (const string &np : newProductions) {
        cout << np << "\n";
    }
}
int main() {
    int n;
    cout << "Enter number of productions: ";
    cin >> n;
    cin.ignore();
    for (int i = 0; i < n; i++) {
        string production;
        cout << "Enter production " << (i + 1) << " (format A->aB|aC|b): ";
        getline(cin, production);
        leftFactor(production);
    }
    return 0;
```

}

Output:


Problems
Output
Debug Console
Terminal
Ports
Code
PS C:\Users\Deepayan\Documents\MAVERICK\labs\5th-sem\cd-lab> cd "c:\Users\Deepayan\Documents\MAVERICK\labs\5th-sem\cd-lab\" ; if ($?) { g++ exp5.cpp -o exp5 } ; if ($?) { .\exp5 }
Enter number of productions: 1
Enter production 1 (format A->aB|aC|b): A->aB|aC|b
A -> aA' | b
A' -> B | C
PS C:\Users\Deepayan\Documents\MAVERICK\labs\5th-sem\cd-lab>